

2.4 Model Selection 모듈 소개

목차

1. 학습/테스트 데이터 세트 분리
 1. 분리하지 않으면 어떻게 되는가 - 과적합 확인
 2. `train_test_split()` 적용
2. 교차 검증(Cross Validation)
 1. 교차 검증이 필요한 이유
 2. K 폴드 교차 검증
 3. K 폴드의 한계 - 레이블 분포 왜곡
 4. Stratified K 폴드
 5. Stratified K 폴드로 성능 평가
3. `cross_val_score()` - 교차 검증 간편 API
4. GridSearchCV - 교차 검증과 하이퍼 파라미터 튜닝을 한 번에
 1. 하이퍼 파라미터 설정
 2. GridSearchCV 수행과 결과 분석
 3. 최적 파라미터와 최고 정확도
 4. `best_estimator_`로 최종 평가
5. 핵심 요약 및 머신러닝 활용 포인트

1. 학습/테스트 데이터 세트 분리

1.1 분리하지 않으면 어떻게 되는가 - 과적합 확인

먼저 학습 데이터로 학습하고 같은 학습 데이터로 예측하면 어떤 일이 벌어지는지 확인해 봅시다.

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

iris = load_iris()
dt_clf = DecisionTreeClassifier()
train_data = iris.data
train_label = iris.target
dt_clf.fit(train_data, train_label)

# 학습 데이터 셋으로 예측 수행
pred = dt_clf.predict(train_data)
print('예측 정확도:', accuracy_score(train_label, pred))
```

줄	코드	상세 설명
5	iris = load_iris()	붓꽃 데이터 150건 전체를 로딩합니다.
6	dt_clf = DecisionTreeClassifier()	결정 트리 분류기를 생성합니다. <code>random_state</code> 를 주지 않았지만, 이 실험에서는 결과가 항상 1.0이므로 영향이 없습니다.
7~8	train_data = iris.data / train_label = iris.target	분리하지 않고 전체 데이터를 학습용으로 사용합니다.
9	dt_clf.fit(train_data, train_label)	150건 전체로 학습합니다. 결정 트리는 기본 설정에서 모든 데이터를 완벽히 분류할 때까지 트리를 깊게 확장합니다.
12	pred = dt_clf.predict(train_data)	학습에 사용한 바로 그 데이터 로 예측합니다. 모델이 이미 답을 외운 상태입니다.
13	accuracy_score(train_label, pred)	정확도가 1.0(100%) 로 나옵니다. 완벽해 보이지만 아무런 의미가 없는 수치 입니다.

▶ 실행 결과

예측 정확도: 1.0

⚠ 결과 해석 - 시험 문제를 미리 보고 시험을 친 것

정확도 100%는 모델이 뛰어나서가 아니라 **이미 정답을 외운 문제를 다시 푼 결과**일 뿐입니다. 이렇게 학습 데이터에만 지나치게 맞춰져 새 데이터에서 성능이 떨어지는 현상을 **과적합(Overfitting)**이라고 합니다. 따라서 예측 성능 평가는 반드시 학습에 사용되지 않은 별도 데이터로 해야 합니다.

1.2 train_test_split() 적용

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
from sklearn.datasets import load_iris
```

```
from sklearn.model_selection import train_test_split

dt_clf = DecisionTreeClassifier( )
iris_data = load_iris()

X_train, X_test, y_train, y_test = train_test_split(iris_data.data, iris_data.target,
                                                    test_size=0.3, random_state=121)
```

줄	코드	상세 설명
9	<code>train_test_split(iris_data.data, iris_data.target, ...)</code>	피처와 레이블을 함께 전달하면 같은 기준으로 섞은 뒤 4개 세트로 나눠 반환합니다.
10	<code>test_size=0.3</code>	테스트 데이터 비율을 30% 로 지정합니다. 150건 × 0.3 = 45건이 테스트용 , 105건이 학습용입니다.
10	<code>random_state=121</code>	분할 결과를 재현 가능하게 고정하는 시드값입니다. 값이 달라지면 정확도도 달라집니다.

```
dt_clf.fit(X_train, y_train)
pred = dt_clf.predict(X_test)
print('예측 정확도: {0:.4f}'.format(accuracy_score(y_test, pred)))
```

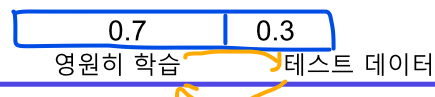
줄	코드	상세 설명
1	<code>dt_clf.fit(X_train, y_train)</code>	학습용 105건으로만 학습합니다.
2	<code>pred = dt_clf.predict(X_test)</code>	모델이 한 번도 보지 못한 45건으로 예측합니다.
3	<code>accuracy_score(y_test, pred)</code>	정확도 0.9556 . 45건 중 43건을 맞혔습니다. 100%였던 앞의 결과와 달리 현실적인 성능 을 보여 줍니다.

▶ 실행 결과

예측 정확도: 0.9556

`train_test.split()`

2. 교차 검증(Cross Validation)



2.1 교차 검증이 필요한 이유

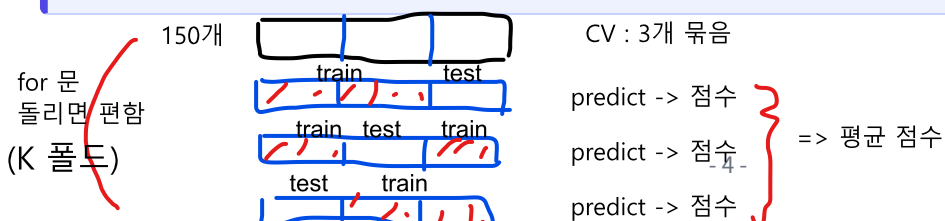
학습/테스트로 한 번만 나누는 방식에도 문제가 있습니다. **어떻게 나누느냐(random_state)에 따라 성능이 크게 달라지고, 모델이 특정 테스트 세트에만 최적화되는 또 다른 과적합**이 발생할 수 있습니다.

교차 검증은 데이터를 여러 세트로 나눠 **학습과 검증을 여러 번 반복**하고 그 결과를 **평균** 내는 방식입니다. 마치 본고사(테스트) 전에 모의고사를 여러 번 치는 것에 비유할 수 있습니다.

■ 데이터 세트의 3분할 구조

교차 검증을 도입하면 데이터는 **학습 → (학습 + 검증) / 테스트**의 3단계로 나뉩니다.

- **학습(Train)** 세트: 모델이 규칙을 배우는 데이터
- **검증(Validation)** 세트: 학습 도중 성능을 점검하고 하이퍼 파라미터를 고르는 데이터
- **테스트(Test)** 세트: 모든 튜닝이 끝난 뒤 **최종 한 번만** 사용하는 데이터



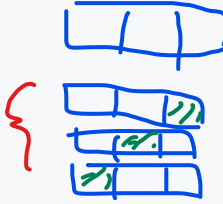
2.2 K 폴드 교차 검증

K 폴드는 데이터를 K개의 동일한 크기로 나눈 뒤, **K번 반복**하면서 매번 다른 폴드 하나를 검증용으로 쓰고 나머지 K-1개를 학습용으로 사용하는 방법입니다.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
from sklearn.model_selection import KFold
import numpy as np

iris = load_iris()
features = iris.data
label = iris.target
dt_clf = DecisionTreeClassifier(random_state=156)
```

KFold class 개수 k 개수만큼



```
# 5개의 폴드 세트로 분리하는 KFold 객체와 폴드 세트별 정확도를 담을 리스트 객체 생성.
kfold = KFold(n_splits=5)
cv_accuracy = []
print('붓꽃 데이터 세트 크기:', features.shape[0])
```

줄	코드	상세 설명
6~8	features = iris.data / label = iris.target	피쳐와 레이블을 각각 변수에 담습니다.
9	dt_clf = DecisionTreeClassifier(random_state=156)	결과 재현을 위해 시드를 고정한 분류기를 생성합니다.
12	kfold = KFold(n_splits=5)	5개의 폴드 로 나누는 KFold 객체를 생성합니다. 150건 ÷ 5 = 폴드당 30건이며, 매 반복마다 학습 120건 / 검증 30건 이 됩니다.
13	cv_accuracy = []	각 폴드의 정확도를 저장할 빈 리스트입니다. 마지막에 평균을 내기 위해 준비합니다.
14	features.shape[0]	전체 데이터 건수 150 을 출력해 폴드 크기 계산의 기준을 확인합니다.

▶ 실행 결과

붓꽃 데이터 세트 크기: 150

5개 나눈 것 가정

```
n_iter = 0
# KFold객체의 split( ) 호출하면 폴드 별 학습용, 검증용 테스트의 로우 인덱스를 array로 반환
for train_index, test_index in kfold.split(features):
    # kfold.split( )으로 반환된 인덱스를 이용하여 학습용, 검증용 테스트 데이터 추출
    X_train, X_test = features[train_index], features[test_index]
    y_train, y_test = label[train_index], label[test_index]
    #학습 및 예측
    dt_clf.fit(X_train, y_train)
    pred = dt_clf.predict(X_test)
    n_iter += 1
    # 반복 시 마다 정확도 측정
    accuracy = np.round(accuracy_score(y_test, pred), 4)
    train_size = X_train.shape[0]
    test_size = X_test.shape[0]
    print('\n#{} 교차 검증 정확도 : {:.1f}, 학습 데이터 크기: {:.2f}, 검증 데이터 크기: {:.3f}'
          .format(n_iter, accuracy, train_size, test_size))
    print('#{} 검증 세트 인덱스: {}'.format(n_iter, test_index))
    cv_accuracy.append(accuracy)
```

인덱스 반환

갯수만큼 나눠진 것 포함된 오브젝트

정확도 점수

원래 답 예측값

120개

30개

리스트에 정확도 append

```
# 개별 iteration별 정확도를 합하여 평균 정확도 계산
print('\n## 평균 검증 정확도:', np.mean(cv_accuracy))
```

줄	코드	상세 설명
1	n_iter = 0	몇 번째 폴드인지 세기 위한 카운터입니다.
4	for train_index, test_index in kfold.split(features):	이 코드의 핵심입니다. <code>split()</code> 은 데이터 자체가 아니라 학습용·검증용 행 인덱스 배열 을 반환하는 제너레이터입니다. 5개 폴드이므로 반복문이 5번 돌 니다.
6~7	X_train, X_test = features[train_index], features[test_index]	반환된 인덱스를 팬시 인덱싱 으로 사용해 실제 데이터를 추출합니다. 레이블 도 동일한 인덱스 로 잘라야 피쳐와 짝이 맞습니다.
9~10	dt_clf.fit(...) / predict(...)	각 폴드마다 새로 학습하고 예측 합니다. 이전 폴드의 학습 내용은 덮어써집니 다.
13	accuracy = np.round(accuracy_score(y_test, pred), 4)	정확도를 소수점 4자리로 반올림합니다.
14~15	train_size / test_size	각 폴드의 학습·검증 데이터 크기를 확인합니다. 모두 120 / 30으로 일정합니 다.
18	print('#{0} 검증 세트 인덱스:{1}'...)	어떤 행이 검증에 쓰였는지 출력합니다. 0~29, 30~59, ... 순차적으로 잘린다는 중요한 사실을 확인할 수 있습니다.
19	cv_accuracy.append(accuracy)	정확도를 리스트에 누적합니다.
22	np.mean(cv_accuracy)	5개 폴드 정확도의 평균 을 최종 성능으로 사용합니다.

▶ 실행 결과

```
#1 교차 검증 정확도 :1.0, 학습 데이터 크기: 120, 검증 데이터 크기: 30
#1 검증 세트 인덱스:[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
 24 25 26 27 28 29]

#2 교차 검증 정확도 :0.9667, 학습 데이터 크기: 120, 검증 데이터 크기: 30
#2 검증 세트 인덱스:[30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53
 54 55 56 57 58 59]

#3 교차 검증 정확도 :0.8667, 학습 데이터 크기: 120, 검증 데이터 크기: 30
#3 검증 세트 인덱스:[60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83
 84 85 86 87 88 89]

#4 교차 검증 정확도 :0.9333, 학습 데이터 크기: 120, 검증 데이터 크기: 30
#4 검증 세트 인덱스:[ 90  91  92  93  94  95  96  97  98  99 100 101 102 103 104 105 106 107
 108 109 110 111 112 113 114 115 116 117 118 119]

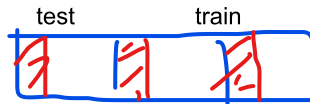
#5 교차 검증 정확도 :0.7333, 학습 데이터 크기: 120, 검증 데이터 크기: 30
#5 검증 세트 인덱스:[120 121 122 123 124 125 126 127 128 129 130 131 132 133 134 135 136 137
 138 139 140 141 142 143 144 145 146 147 148 149]

## 평균 검증 정확도: 0.9
```

■ 결과 해석

폴드별 정확도가 **1.0에서 0.7333까지** 큰 폭으로 흔들립니다. 특히 5번째 폴드가 0.7333으로 유독 낮습니다. 이는 **붓꽃 데이터가 레이블 순으로 정렬되어** 있어(2.3장에서 확인) 각 폴드의 레이블 구성이 편향되었기 때문입니다. 평균 0.9라는 값도 이런 불안정한 결과들의 평균이므로 신뢰하기 어렵습니다. 다음 절에서 그 원인을 직접 확인합니다.

2.3 K 폴드의 한계 - 레이블 분포 왜곡



150. k = 3으로 나눔.

```
import pandas as pd
```

```
iris = load_iris()
```

```
iris_df = pd.DataFrame(data=iris.data, columns=iris.feature_names)
```

```
iris_df['label']=iris.target
```

```
iris_df['label'].value_counts()
```

iris는

여기는
다 0

다 1

다 2

1,2 공부해서 시험은 0으로 봄..
=> 답을 보고 나누겠다.(Stratified K 폴드)

줄	코드	상세 설명
5~6	<pre>iris_df = pd.DataFrame(...) / iris_df['label']=iris.target</pre>	레이블 분포를 판다스로 확인하기 위해 DataFrame으로 변환합니다.
7	<pre>iris_df['label'].value_counts()</pre>	세 클래스가 각각 50건씩 완벽히 균등합니다. 전체적으로는 문제가 없어 보입니다.

▶ 실행 결과

```
label  
0    50  
1    50  
2    50  
Name: count, dtype: int64
```

```
kfold = KFold(n_splits=3)  
# kfold.split(X)는 폴드 세트를 3번 반복할 때마다 달라지는 학습/테스트 용 데이터 로우 인덱스 번호 반환.  
n_iter = 0  
for train_index, test_index in kfold.split(iris_df):  
    n_iter += 1  
    label_train= iris_df['label'].iloc[train_index]  
    label_test= iris_df['label'].iloc[test_index]  
    print('## 교차 검증: {0}'.format(n_iter))  
    print('학습 레이블 데이터 분포:\n', label_train.value_counts())  
    print('검증 레이블 데이터 분포:\n', label_test.value_counts())
```

줄	코드	상세 설명
1	<pre>kfold = KFold(n_splits=3)</pre>	문제를 극명하게 드러내기 위해 폴드를 3개로 설정합니다. 150건 ÷ 3 = 폴드 당 50건이며, 이는 클래스당 개수와 정확히 일치합니다.
4	<pre>for train_index, test_index in kfold.split(iris_df):</pre>	DataFrame을 넣어도 행 인덱스 배열이 동일하게 반환됩니다.
6~7	<pre>iris_df['label'].iloc[train_index]</pre>	반환된 인덱스는 위치 기반이므로 iloc 으로 접근해야 합니다.
9~10	<pre>label_train.value_counts()</pre>	학습·검증 세트의 레이블 분포를 출력합니다. 결과가 충격적입니다.

▶ 실행 결과

```
## 교차 검증: 1  
학습 레이블 데이터 분포:  
label  
1    50  
2    50  
검증 레이블 데이터 분포:  
label  
0    50  
## 교차 검증: 2
```

```

학습 레이블 데이터 분포:
label
0    50
2    50
검증 레이블 데이터 분포:
label
1    50
## 교차 검증: 3
학습 레이블 데이터 분포:
label
0    50
1    50
검증 레이블 데이터 분포:
label
2    50

```

⚠ 치명적인 문제 - 정확도가 0이 되는 이유

첫 번째 폴드를 보면 학습 데이터에는 0번 클래스가 전혀 없고, 검증 데이터는 전부 0번 클래스입니다. 배우지 않은 것을 시험 보는 상황이므로 정확도는 0%가 됩니다. 데이터가 레이블 순으로 정렬되어 있는데 K 폴드가 순차적으로 잘랐기 때문입니다. 이 문제를 해결하는 것이 **Stratified K 폴드**입니다.

2.4 Stratified K 폴드

답 쏠려있는 것 확인하고 쓰기. 막 쓰지 말고.

Stratified K 폴드는 원본 데이터의 레이블 분포를 각 폴드에 그대로 유지하면서 나누는 방식입니다.

```

from sklearn.model_selection import StratifiedKFold

skf = StratifiedKFold(n_splits=3)
n_iter=0

for train_index, test_index in skf.split(iris_df, iris_df['label']):
    n_iter += 1
    label_train= iris_df['label'].iloc[train_index]
    label_test= iris_df['label'].iloc[test_index]
    print('## 교차 검증: {0}'.format(n_iter))
    print('학습 레이블 데이터 분포:\n', label_train.value_counts())
    print('검증 레이블 데이터 분포:\n', label_test.value_counts())

```

줄	코드	상세 설명
3	skf = StratifiedKFold(n_splits=3)	KFold와 동일하게 폴드 수를 지정합니다. 클래스 이름만 다릅니다.
6	skf.split(iris_df, iris_df['label'])	가장 중요한 차이점 입니다. KFold의 <code>split()</code> 은 피쳐만 받았지만, StratifiedKFold는 레이블 데이터를 반드시 두 번째 인자로 전달해야 합니다. 레이블 분포를 알아야 균등하게 나눌 수 있기 때문입니다.
8~12	분포 출력	학습 세트에 세 클래스가 33/33/34로, 검증 세트에도 17/17/16으로 고르게 분포 합니다.

▶ 실행 결과

```

## 교차 검증: 1
학습 레이블 데이터 분포:
label
2    34
0    33
1    33
검증 레이블 데이터 분포:

```

```

label
0    17
1    17
2    16
## 교차 검증: 2
학습 레이블 데이터 분포:
label
1    34
0    33
2    33
검증 레이블 데이터 분포:
label
0    17
2    17
1    16
## 교차 검증: 3
학습 레이블 데이터 분포:
label
0    34
1    33
2    33
검증 레이블 데이터 분포:
label
1    17
2    17
0    16

```

■ 결과 해석

모든 폴드에서 세 클래스가 원본과 같은 1:1:1 비율로 유지됩니다. 앞의 K 폴드처럼 특정 클래스가 통째로 빠지는 일이 사라졌습니다. 특히 클래스가 불균형한 실무 데이터(예: 사기 거래 탐지에서 사기가 0.1%)에서는 Stratified K 폴드가 **사실상 필수**입니다.

2.5 Stratified K 폴드로 성능 평가

```

dt_clf = DecisionTreeClassifier(random_state=156)

skfold = StratifiedKFold(n_splits=3)
n_iter=0
cv_accuracy=[]

# StratifiedKFold의 split( ) 호출시 반드시 레이블 데이터 셋도 추가 입력 필요
for train_index, test_index in skfold.split(features, label):
    # split( )으로 반환된 인덱스를 이용하여 학습용, 검증용 테스트 데이터 추출
    X_train, X_test = features[train_index], features[test_index]
    y_train, y_test = label[train_index], label[test_index]
    #학습 및 예측
    dt_clf.fit(X_train, y_train)
    pred = dt_clf.predict(X_test)

    # 반복 시 마다 정확도 측정
    n_iter += 1
    accuracy = np.round(accuracy_score(y_test, pred), 4)
    train_size = X_train.shape[0]
    test_size = X_test.shape[0]
    print('\n#{0} 교차 검증 정확도 :{1}, 학습 데이터 크기: {2}, 검증 데이터 크기: {3}'
          .format(n_iter, accuracy, train_size, test_size))
    print('#{0} 검증 세트 인덱스:{1}'.format(n_iter, test_index))

```

```
cv_accuracy.append(accuracy)
```

```
# 교차 검증별 정확도 및 평균 정확도 계산
print('\n## 교차 검증별 정확도:', np.round(cv_accuracy, 4))
print('## 평균 검증 정확도:', np.round(np.mean(cv_accuracy), 4))
```

줄	코드	상세 설명
8	skfold.split(features, label)	이번에는 ndarray 인 features와 label을 전달합니다. 레이블을 함께 넘기는 것이 핵심입니다.
10~11	features[train_index] 등	ndarray이므로 iloc 없이 바로 팬시 인덱싱 이 가능합니다.
19	train_size / test_size	3폴드이므로 학습 100건 / 검증 50건 이 됩니다.
23	print('#{0} 검증 세트 인덱스:{1}')	검증 인덱스를 보면 0~16, 50~66, 100~115처럼 세 구간에서 골고루 뽑혔음을 알 수 있습니다. K 폴드가 0~49를 통째로 뽑았던 것과 대조적입니다.
27~28	교차 검증별 / 평균 정확도	0.98, 0.94, 0.98로 편차가 매우 작고 평균도 0.9667로 크게 향상되었습니다.

▶ 실행 결과

```
#1 교차 검증 정확도 :0.98, 학습 데이터 크기: 100, 검증 데이터 크기: 50
#1 검증 세트 인덱스:[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 50
 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 100 101
102 103 104 105 106 107 108 109 110 111 112 113 114 115]

#2 교차 검증 정확도 :0.94, 학습 데이터 크기: 100, 검증 데이터 크기: 50
#2 검증 세트 인덱스:[ 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 67
 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 116 117 118
119 120 121 122 123 124 125 126 127 128 129 130 131 132]

#3 교차 검증 정확도 :0.98, 학습 데이터 크기: 100, 검증 데이터 크기: 50
#3 검증 세트 인덱스:[ 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 83 84
 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 133 134 135
136 137 138 139 140 141 142 143 144 145 146 147 148 149]

## 교차 검증별 정확도: [0.98 0.94 0.98]
## 평균 검증 정확도: 0.9667
```

■ K 폴드 vs Stratified K 폴드 비교

구분	K 폴드 (5분할)	Stratified K 폴드 (3분할)
폴드별 정확도	1.0 / 0.9667 / 0.8667 / 0.9333 / 0.7333	0.98 / 0.94 / 0.98
편차	매우 큼 (최대 0.27 차이)	매우 작음 (최대 0.04 차이)
평균	0.9	0.9667
split 인자	피쳐만	피쳐 + 레이블 필수

분류(Classification)에서는 반드시 Stratified K 폴드를 사용해야 합니다. 다만 **회귀(Regression)**는 레이블이 연속값이라 분포를 나눌 수 없으므로 **일반 K 폴드만 지원**됩니다.

간단하게 교차검증 점수 추출할 때 사용

3. cross_val_score() - 교차 검증 간편 API

앞의 코드는 폴드 분리, 반복문, 학습·예측, 정확도 누적을 직접 작성해야 했습니다. `cross_val_score()` 는 이 모든 과정을 한 줄로 처리해 줍니다.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score, cross_validate
from sklearn.datasets import load_iris

iris_data = load_iris()  # 데이터
dt_clf = DecisionTreeClassifier(random_state=156)  # 모델
data = iris_data.data
label = iris_data.target  # 답
```

학습
↓
예측
↓
검증

성능 지표는 정확도 (accuracy), 교차 검증 세트는 3개

```
scores = cross_val_score(dt_clf, data, label, scoring='accuracy', cv=3)
print('교차 검증별 정확도:', np.round(scores, 4))
print('평균 검증 정확도:', np.round(np.mean(scores), 4))
```

학습, 예측, 검증, 점수 출력 한 번에 점수를 뽑는 기능이지 모델이 없음. 모델 선택할 때 사용(어떤 모델의 점수가 좋은지)

줄	코드	상세 설명
12	<code>cross_val_score(dt_clf, data, label, scoring='accuracy', cv=3)</code>	인자 순서는 ① estimator (모델 객체), ② 피쳐 데이터, ③ 레이블 데이터, ④ 평가 지표 , ⑤ 폴드 수입입니다. 반환값은 각 폴드의 평가 점수 배열입니다.
12	<code>scoring='accuracy'</code>	성능 평가 지표를 지정합니다. 'precision', 'recall', 'f1', 'roc_auc', 회귀는 'neg_mean_squared_error' 등을 쓸 수 있습니다.
12	<code>cv=3</code>	교차 검증 폴드 수입입니다. 중요한 점은 estimator가 Classifier면 자동으로 Stratified K 폴드가 적용된다는 것 입니다(회귀는 일반 K 폴드). 그래서 앞 절의 StratifiedKFold 결과와 정확히 같은 값이 나옵니다.
13~14	<code>np.round(scores, 4) / np.mean(scores)</code>	폴드별 점수와 평균을 출력합니다. 결과가 [0.98, 0.94, 0.98], 평균 0.9667로 2.5절과 완벽히 일치합니다.

▶ 실행 결과

```
교차 검증별 정확도: [0.98 0.94 0.98]
평균 검증 정확도: 0.9667
```

■ cross_val_score vs cross_validate

`cross_val_score()` 는 하나의 평가 지표만 반환하지만, `cross_validate()` 는 여러 지표를 동시에 평가하고 학습 시간·평가 시간까지 디렉터리로 반환합니다. 여러 지표를 함께 봐야 할 때 유용합니다.

4. GridSearchCV - 교차 검증과 하이퍼 파라미터 튜닝을 한 번에

하이퍼 파라미터는 사람이 직접 지정해야 하는 알고리즘의 설정값(트리 깊이, 분할 최소 샘플 수 등)으로, 모델 성능을 크게 좌우합니다. `GridSearchCV` 는 지정한 하이퍼 파라미터 값들을 모든 조합으로 순차 적용하면서 교차 검증까지 수행해 최적 조합을 자동으로 찾아 줍니다.

4.1 하이퍼 파라미터 설정

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV

# 데이터를 로딩하고 학습데이터와 테스트 데이터 분리
iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(iris_data.data, iris_data.target,
                                                    test_size=0.2, random_state=121)

dtree = DecisionTreeClassifier()

### parameter 들을 dictionary 형태로 설정
parameters = {'max_depth':[1,2,3], 'min_samples_split':[2,3]}
```

줄	코드	상세 설명
7~8	<code>train_test_split(..., test_size=0.2, random_state=121)</code>	150건을 학습 120건 / 테스트 30건 으로 나눕니다. 하이퍼 파라미터 탐색은 학습 데이터 안에서만 이뤄지고, 테스트 데이터는 마지막에 한 번만 사용됩니다.
9	<code>dtree = DecisionTreeClassifier()</code>	튜닝 대상 모델을 생성합니다. 하이퍼 파라미터는 여기서 지정하지 않고 GridSearchCV가 대신 넣어 줍니다.
12	<code>parameters = {'max_depth':[1,2,3], 'min_samples_split':[2,3]}</code>	탐색할 하이퍼 파라미터를 딕셔너리 로 정의합니다. Key 는 파라미터 이름(문자열) , Value 는 시도할 값들의 리스트 입니다. <code>max_depth</code> : 트리의 최대 깊이. 작을수록 단순한 모델 <code>min_samples_split</code> : 노드를 분할하기 위한 최소 샘플 수. 클수록 분할이 억제됨 조합 수는 $3 \times 2 = 6$ 가지입니다.

4.2 GridSearchCV 수행과 결과 분석

```
import pandas as pd

# param_grid의 하이퍼 파라미터들을 3개의 train, test set fold 로 나누어서 테스트 수행 설정.
### refit=True 가 default 임. True이면 가장 좋은 파라미터 설정으로 재 학습 시킴.
grid_dtree = GridSearchCV(dtree, param_grid=parameters, cv=3, refit=True)

# 붓꽃 Train 데이터로 param_grid의 하이퍼 파라미터들을 순차적으로 학습/평가 .
grid_dtree.fit(X_train, y_train)

# GridSearchCV 결과 추출하여 DataFrame으로 변환
scores_df = pd.DataFrame(grid_dtree.cv_results_)
scores_df[['params', 'mean_test_score', 'rank_test_score', \
           'split0_test_score', 'split1_test_score', 'split2_test_score']]
```

줄	코드	상세 설명
5	<code>GridSearchCV(dtree, param_grid=parameters, cv=3, refit=True)</code>	① dtree : 튜닝할 estimator ② param_grid : 탐색할 파라미터 딕셔너리 ③ cv=3 : 각 조합을 3폴드 교차 검증 으로 평가 ④ refit=True (기본값): 탐색이 끝나면 최적 조합으로 전체 학습 데이터를 다시 학습 해 둡니다.

8	<code>grid_dtree.fit(X_train, y_train)</code>	실제 탐색을 수행합니다. 내부적으로 6개 조합 × 3폴드 = 18번 학습·평가가 이뤄집니다. 조합이 많아지면 시간이 급격히 늘어나므로 주의해야 합니다.
11	<code>scores_df = pd.DataFrame(grid_dtree.cv_results_)</code>	<code>cv_results_</code> 는 모든 조합의 상세 결과가 담긴 딕셔너리입니다. DataFrame으로 변환하면 표 형태로 비교하기 좋습니다.
12~13	<code>scores_df[['params', 'mean_test_score', ...]]</code>	주요 컬럼만 선택합니다. <code>params</code> : 적용된 파라미터 조합 <code>mean_test_score</code> : 3개 폴드 점수의 평균 (최적 판단 기준) <code>rank_test_score</code> : 평균 점수 기준 순위 (1이 최고) <code>split0~2_test_score</code> : 폴드별 개별 점수

▶ 실행 결과

	params	mean_test_score	rank_test_score	split0_test_score	split1_test_score	split2_test_score
0	<code>{'max_depth': 1, 'min_samples_split': 2}</code>	0.700000	5	0.700	0.7	0.70
1	<code>{'max_depth': 1, 'min_samples_split': 3}</code>	0.700000	5	0.700	0.7	0.70
2	<code>{'max_depth': 2, 'min_samples_split': 2}</code>	0.958333	3	0.925	1.0	0.95
3	<code>{'max_depth': 2, 'min_samples_split': 3}</code>	0.958333	3	0.925	1.0	0.95
4	<code>{'max_depth': 3, 'min_samples_split': 2}</code>	0.975000	1	0.975	1.0	0.95
5	<code>{'max_depth': 3, 'min_samples_split': 3}</code>	0.975000	1	0.975	1.0	0.95

■ 결과 해석

- max_depth=1**(인덱스 0, 1)은 정확도가 0.7에 그칩니다. 깊이 1짜리 트리는 **단 한 번만 분기**하므로 3개 클래스를 구분할 수 없습니다(setosa만 분리 가능). 전형적인 **과소적합(Underfitting)**입니다.
- max_depth가 2 → 3**으로 커지면서 0.9583 → 0.975로 성능이 향상됩니다.
- min_samples_split은 2든 3이든 결과가 동일합니다.** 이 데이터에서는 영향을 주지 않는 파라미터임을 알 수 있습니다.
- rank가 **1인 조합이 두 개인** 이유는 점수가 완전히 같기 때문이며, 이 경우 먼저 나온 조합이 선택됩니다.

4.3 최적 파라미터와 최고 정확도

```
print('GridSearchCV 최적 파라미터:', grid_dtree.best_params_)
print('GridSearchCV 최고 정확도: {0:.4f}'.format(grid_dtree.best_score_))
```

줄	코드	상세 설명
1	<code>grid_dtree.best_params_</code>	가장 좋은 성능을 낸 파라미터 조합 을 딕셔너리로 반환합니다. 속성명 끝에 언더스코어(_) 가 붙은 것은 사이킷런에서 학습 후에 생성되는 속성 이라는 관례 표시입니다.
2	<code>grid_dtree.best_score_</code>	그때의 교차 검증 평균 정확도 입니다. 위 표에서 rank 1인 행의 <code>mean_test_score</code> 와 동일한 0.9750입니다.

▶ 실행 결과

```
GridSearchCV 최적 파라미터: {'max_depth': 3, 'min_samples_split': 2}
GridSearchCV 최고 정확도: 0.9750
```

4.4 best_estimator_로 최종 평가

```
# GridSearchCV의 refit으로 이미 학습이 된 estimator 반환
estimator = grid_dtree.best_estimator_
```

```
# GridSearchCV의 best_estimator_는 이미 최적 하이퍼 파라미터로 학습이 됨
pred = estimator.predict(X_test)
print('테스트 데이터 세트 정확도: {:.4f}'.format(accuracy_score(y_test, pred)))
```

줄	코드	상세 설명
2	estimator = grid_dtree.best_estimator_	<code>refit=True</code> 덕분에 최적 파라미터로 전체 학습 데이터를 이미 학습해 둔 모델 이 반환됩니다. 따로 <code>fit()</code> 을 다시 호출할 필요가 없습니다.
5	pred = estimator.predict(X_test)	지금까지 한 번도 사용하지 않은 테스트 데이터 30건 으로 최종 예측을 수행합니다.
6	accuracy_score(y_test, pred)	최종 정확도 0.9667 . 30건 중 29건을 맞혔습니다.

▶ 실행 결과

테스트 데이터 세트 정확도: 0.9667

■ 결과 해석 - 교차 검증 점수와 테스트 점수의 차이

교차 검증 평균은 0.9750, 최종 테스트는 0.9667로 **약간 낮습니다**. 이는 정상적인 현상입니다. 교차 검증 점수는 **여러 조합 중 가장 좋은 값을 고른 결과**라 다소 낙관적으로 나오는 경향이 있기 때문입니다. **모델의 실제 성능은 테스트 세트 점수로 보고해야** 합니다.

■ GridSearchCV 사용의 표준 패턴

① `train_test_split` 으로 **테스트 세트를 먼저 떼어 놓는다** → ② 학습 데이터로만 `GridSearchCV.fit()` 수행 → ③ `best_params_`, `best_score_` 로 튜닝 결과 확인 → ④ `best_estimator_` 로 **테스트 세트에서 최종 평가**. 이 순서를 지키지 않고 테스트 데이터까지 튜닝에 사용하면 **성능이 부풀려집니다**.

5. 핵심 요약 및 머신러닝 활용 포인트

5.1 핵심 API 요약표

API	역할	핵심 인자 / 속성	주의 사항
<code>train_test_split()</code>	학습/테스트 분리	<code>test_size</code> , <code>random_state</code> , <code>stratify</code>	반환 순서 <code>X_train</code> , <code>X_test</code> , <code>y_train</code> , <code>y_test</code>
<code>KFold()</code>	K개 폴드 분리	<code>n_splits</code> <code>split(X)</code>	레이블 분포를 고려하지 않음. 분류에는 부적합
<code>StratifiedKFold()</code>	레이블 분포 유지 분리	<code>n_splits</code> <code>split(X, y)</code>	레이블을 반드시 함께 전달 . 회귀에는 사용 불가
<code>cross_val_score()</code>	교차 검증 한 번에 수행	<code>estimator</code> , <code>X</code> , <code>y</code> , <code>scoring</code> , <code>cv</code>	Classifier면 Stratified가 자동 적용
<code>cross_validate()</code>	다중 지표 교차 검증	<code>scoring</code> 에 리스트 전달 가능	학습·평가 소요 시간도 반환
<code>GridSearchCV()</code>	하이퍼 파라미터 튜닝 + 교차 검증	<code>param_grid</code> , <code>cv</code> , <code>refit</code> , <code>scoring</code> <code>best_params_</code> , <code>best_score_</code> , <code>best_estimator_</code> , <code>cv_results_</code>	조합 수 × 폴드 수만큼 학습하므로 시간 급증 주의

5.2 교차 검증 방식 선택 가이드

상황	권장 방식
분류 문제 (일반)	<code>StratifiedKFold</code> 또는 <code>cross_val_score</code> (자동 적용)
회귀 문제	<code>KFold</code> (레이블 분포 개념이 없음)
클래스 불균형이 심한 분류	<code>StratifiedKFold</code> 필수
시계열 데이터	<code>TimeSeriesSplit</code> (과거로 학습, 미래로 검증)
하이퍼 파라미터 탐색	<code>GridSearchCV</code> (전수 탐색) 또는 <code>RandomizedSearchCV</code> (무작위 탐색, 조합이 많을 때)

5.3 머신러닝 활용 포인트

- **학습 데이터로 평가하지 않기** : 정확도 100%가 나오면 가장 먼저 데이터 분리를 의심해야 합니다.
- **단일 분할을 믿지 않기** : `random_state`에 따라 성능이 흔들린다면 교차 검증으로 안정적인 추정치를 구하세요.
- **분류는 Stratified** : 특히 불균형 데이터에서 일반 K 폴드는 폴드별 성능을 심하게 왜곡시킵니다.
- **테스트 세트는 마지막에 한 번만** : 튜닝 과정에서 테스트 데이터를 반복해서 들여다보면 그 세트에 과적합됩니다.
- **탐색 범위를 넓게 시작해 좁혀 가기** : 처음부터 촘촘한 그리드를 쓰면 시간만 소모됩니다. 성능이 좋은 구간을 찾은 뒤 세밀하게 탐색하세요.

⚠ 안정화 버전 참고 (scikit-learn 1.7.x 기준)

- 본 강의 API는 오랜 기간 안정적으로 유지되어 왔으며 **사용법 변경이 없습니다**.
- `GridSearchCV` 에 `n_jobs=-1` 을 주면 **모든 CPU 코어를 사용해** 탐색 속도를 크게 높일 수 있습니다.
- 조합 수가 매우 많을 때는 `RandomizedSearchCV` 나 `HalvingGridSearchCV` (연속 반감 탐색)를 검토하세요.
- `cv_results_` 의 컬럼 구성은 버전에 따라 일부 항목이 추가될 수 있으나, 본문에서 사용한 주요 컬럼은 동일합니다.